به نام خدا

فاز دوم پروژه درس تحقیق در عملیات 1

استاد کوروش عشقی

اعضای گروه:

صبا یوسف زاده

روزبه ملک احمدی

مونا اصغرزاده

قسمت الف)

در این قسمت مدلسازی ای که در فاز یک انجام داده بودیم را در پایتون به کمک کتابخانه pyomo پیاده سازی میکنیم و در نهایت توسط glpk حل میکنیم.

در ابتدا متغیرهای تصمیم گیری را مینویسیم سپس محدودیت ها را تعریف میکنیم و در انتها تابع هدف را تشکیل میدهیم.

مدلسازی فاز یک ما مشکلاتی داشت که مانع از جواب صحیح میشد. پس ما مجبور به تغییر مدلسازی فاز یک شدیم.

پی دی اف تصحیح شده مدلسازی در زیپ قرار دارد.

همچنین در انتها از کد ران گرفته و در فایل ذخیره میکنیم.

قسمت ب)

در این قسمت برای اضافه کردن تخفیف به تابع هدف مجبور به استفاده از خطی سازی شدیم چون غیر خطی بود.

نتیجه ران کردن این کد نیز در فایل قابل مشاهده است.

قسمت د)

با استفاده از حلقه در پایتون، هزینه تبدیل فلز به الیاژ را ده تا ده تا افزایش میدهیم و هرگاه جمع مقدار الیاژ تولید شده در کارخانه به صفر برسد، نشان میدهد که دیگر تولید به صرفه نیست و فقط باید از کارخانه های رقیب بخریم.

```
280  # حلقه تحلیل حساسیت بخش د
281  while True:
282  # حل مدل
283      results = solver.solve(model)
284      if model.AP['A','0'].value+model.AP['B','0'].value==0:
285          threshold_cost = model.cost_change.value
286          break
287
288  # افزایش cost به مقدار 10 (واحد step)
289      model.cost_change.set_value(model.cost_change.value + 10)
290
291  # خروجی
292  print(f"هزینه بحرانی : {threshold_cost}")
293  model.cost_change=250
294
295
296  #=============================================================================
```

با استفاده از کد زده شده، این مقدار بحرانی 400 در می اید.

قسمت ه)

در این قسمت نیز مانند قسمت قبلی با حلقه سعی میکنیم هزینه قراردادی که با ان دیگر خرید از کارخانه مربوطه منطقی نمیباشد را پیدا کنیم.

هزینه بحرانی برای قرارداد 1 ، 450120

هزینه بحرانی برای قرارداد 2 ، 240090

```
297    # حلقه تحلیل حساسیت بخش ه
298     # قرارداد 1
299    while True:
300    # حل مدل
301        results = solver.solve(model)
302        if model.AB['A','1'].value+model.AB['B','1'].value==0:
303            threshold_cost = model.deal1.value
304            break
305
306
307    # افزایش cost به مقدار 10 (واحد step)
308        model.deal1.set_value(model.deal1.value +10000)
309        print(model.deal1.value)
310
311    # خروجی
312    print(f"هزینه بحرانی : {threshold_cost}")
313    model.deal1=120
314    #-----------------------------------------------------------------
315    # قرارداد 2
316    while True:
317    # حل مدل
318        results = solver.solve(model)
319        if model.AB['A','2'].value+model.AB['B','2'].value==0:
320            threshold_cost = model.deal2.value
321            break
322
323
324    # افزایش cost به مقدار 10 (واحد step)
325        model.deal2.set_value(model.deal2.value +10000)
326        print(model.deal2.value)
327
328    # خروجی
329    print(f"هزینه بحرانی : {threshold_cost}")
330    model.deal1=90
```

کد نوشته شده شامل دو حلقه است که مقدار هزینه قراردادها را اینقد زیاد میکنند که دیگر متغیر خرید از ان کارخانه صفر بشود و خرید به صرفه نباشد.

قسمت و)

در این قسمت نیز با کمک حلقه سعی در پیدا کردن نقطه ای را داریم که مقادیر بهینه تغییر کنند.

برای هر هزینه خرید الیاژ از کارخانه 1 و 2 دو حلقه ران میکنیم، یکی برای پیدا کردن حد بالا و دیگری برای پیدا کردن پایین.

```
    initial_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    new_basis= {v.name: value(v) for v in model.component_data_objects(Var)}
    while new_basis==initial_basis:
        model.factory2_B.set_value(model.factory2_B.value+1 )
        solver.solve(model)
        new_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    print(model.factory2_B.value) #=> 533,541
    initial_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    new_basis= {v.name: value(v) for v in model.component_data_objects(Var)}
    while new_basis==initial_basis:
        model.factory2_B.set_value(model.factory2_B.value+1 )
        solver.solve(model)
        new_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    print(model.factory2_B.value) #=> 533,541
    #---------------------------------------------------------------
    1 در مارخانه B حد بالا و پایین برای قیمت الیاژ #
    initial_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    new_basis= {v.name: value(v) for v in model.component_data_objects(Var)}
    while new_basis==initial_basis:
        model.factory1_B.set_value(model.factory1_B.value+1 )
        solver.solve(model)
        new_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    print(model.factory1_B.value) #=> 519,521

    initial_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    new_basis= {v.name: value(v) for v in model.component_data_objects(Var)}
    while new_basis==initial_basis:
        model.factory1_B.set_value(model.factory1_B.value+1 )
        solver.solve(model)
        new_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    print(model.factory1_B.value) #=> 519,521
    #---------------------------------------------------------------
```

```
    2 در مارخانه A حد بالا و پایین برای قیمت الیاژ #
    initial_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    new_basis= {v.name: value(v) for v in model.component_data_objects(Var)}
    while new_basis==initial_basis:
        model.factory2_A.set_value(model.factory2_A.value+1 )
        solver.solve(model)
        new_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    print(model.factory2_A.value) #=> 389,391
    while new_basis==initial_basis:
        model.factory2_A.set_value(model.factory2_A.value-1 )
        solver.solve(model)
        new_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    print(model.factory2_A.value) #=> 389,391
    #---------------------------------------------------------------
    1 در مارخانه Aالیاژ حد بالا و پایین برای قیمت #
    initial_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    new_basis= {v.name: value(v) for v in model.component_data_objects(Var)}
    while new_basis==initial_basis:
        model.factory1_A.set_value(model.factory1_A.value+1 )
        solver.solve(model)
        new_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    print(model.factory1_A.value) #=> 374,376

    initial_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    new_basis= {v.name: value(v) for v in model.component_data_objects(Var)}
    while new_basis==initial_basis:
        model.factory1_A.set_value(model.factory1_A.value+1 )
        solver.solve(model)
        new_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
    print(model.factory1_A.value) #=> 374,376
    #---------------------------------------------------------------
```

که حد بالا و پایین به ترتیب برای :

کارخانه A و الیاژ 1 : 374 و 376

کارخانه B و الیاژ 1: 519 و 521

کارخانه A و الیاژ 2 : 389 و 391

کارخانه B و الیاژ 2 : 533 و 541

به دست می اید.

قسمت ز)

مقدار ضریب هزینه تبدیل فلز به الیاژ که در ابتدای مسئله 250 بود را تغییر داده و تاثیرش را بر تابع هدف نشان میدهیم

هزینه ها: = C

$$45O_1 + 65O_2 + 70O_3$$

$$250(AP_{A0} + AP_{B0})$$

$$375AB_{A1} + 520AB_{B1} + 390AB_{A2} + 540AB_{B2}$$

$$120C_1 + 90C_2$$

$$\sum_F \sum_S (CCS_{FS} + ES_{FS}) * TCS_{FS}$$

$$\sum_S \sum_C (CCC_{SC} + EC_{SC}) * TCC_{SC}$$

درآمد ها: = R

$$\sum_i \sum_C (AIC_{iTC} + AIC_{iEC}) * ASP_{iC}$$

Max Z = R - C

```python
# بخش ز
# بررسی اثر تغییرات هزینه تبدیل الیاژ به فلز بر تابع هدف
import matplotlib.pyplot as plt
from pyomo.environ import *

# فرض می‌کنیم model قبلاً تعریف شده است و شامل متغیر cost_change می‌باشد.
# مقدار اولیه cost_change
model.cost_change.set_value(0)

# ایجاد یک لیست برای ذخیره مقادیر تابع هدف
objective_values = []
cost_change_values = []

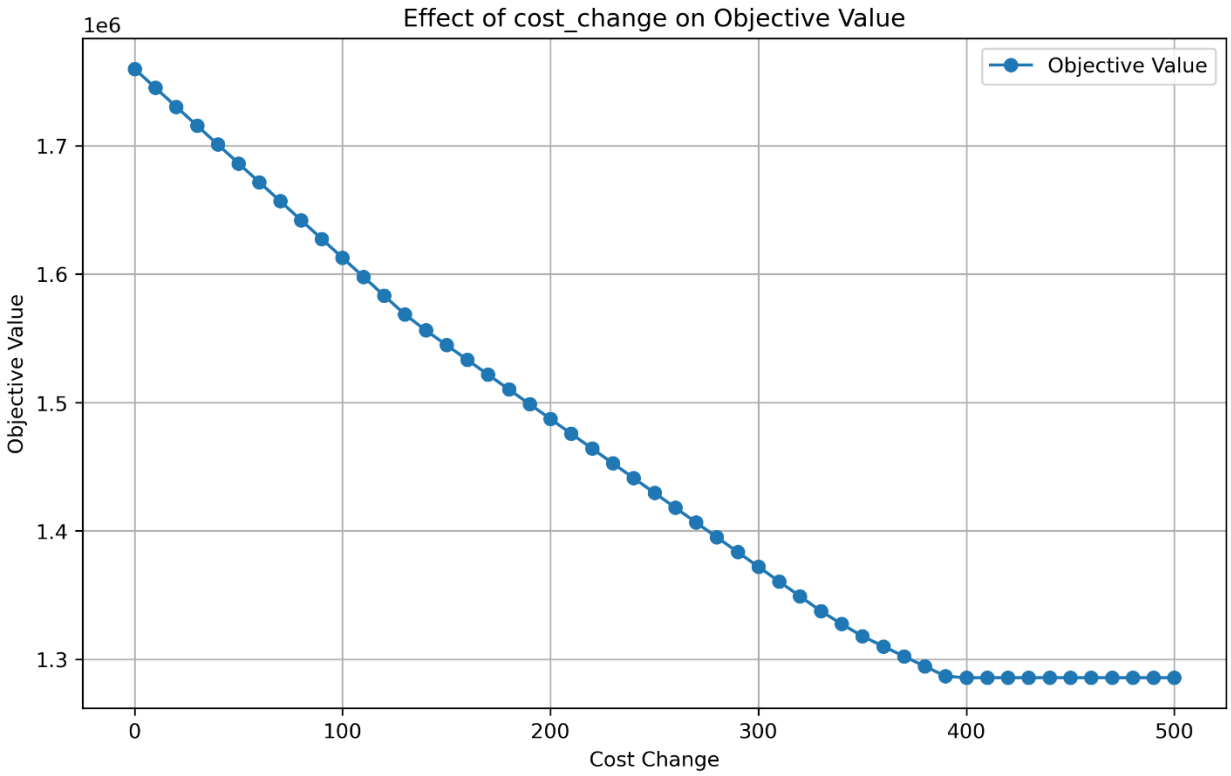
# بازه تغییرات cost_change
cost_change_range = range(0, 501, 10)  # از 0 تا 500 با گام 10

for cost in cost_change_range:
# تنظیم مقدار جدید برای cost_change
    model.cost_change.set_value(cost)

# حل مدل
    results = solver.solve(model)

# ذخیره مقدار تابع هدف
    if results.solver.termination_condition == TerminationCondition.optimal:
        objective_values.append(value(model.Profit))
        cost_change_values.append(cost)
    else:
        print(f"مدل با cost_change={cost} حل نشد.")
        break

# رسم نمودار
plt.figure(figsize=(10, 6))
plt.plot(cost_change_values, objective_values, marker='o', label='Objective Value')
plt.xlabel('Cost Change')
plt.ylabel('Objective Value')
plt.title('Effect of cost_change on Objective Value')
plt.grid()
plt.legend()
plt.savefig('D:objective_vs_cost_change.png', dpi=300, bbox_inches='tight')  # تنظیمات ذخیره
plt.show()
```



همچنین مقدار تابع هدف را با تغییر یکی از مقادیر سمت راست باید بررسی کنیم. ما حداکثر تعداد کانتینر ارسال شده به انبار تهران که در ابتدا برابر با 65 بود را تغییر دادیم.

مجموع حمل و نقل:

تهران:

$$\sum_{F=0}^{2} CCS_{FT} + ES_{FT} \leq 65$$

$$\sum_{F=0}^{2} CCS_{FT} + ES_{FT} \geq 20$$

اصفهان:

```python
#--------------------------------------------------------------------------
# تغییر ماکس حداکثر مجموع حمل و نقل به تهران
['transport_limits['T']['max لیست مقادیر #
transport_limits_values = list(range(50, 101, 5))  # بازه‌ای از 50 تا 100 با گام 5
objective_values = []

['transport_limits['T']['max حلقه برای تغییر مقدار # تعریف پارامتر برای محدودیت حمل‌ونقل و حل مدل
model.transport_limit = Param(initialize=65, mutable=True)

# تعریف قید با استفاده از پارامتر
model.transport_constraints.add(
    sum(model.CCS[f, 'T']  for f in model.production_facilities) <= model.transport_limit
)

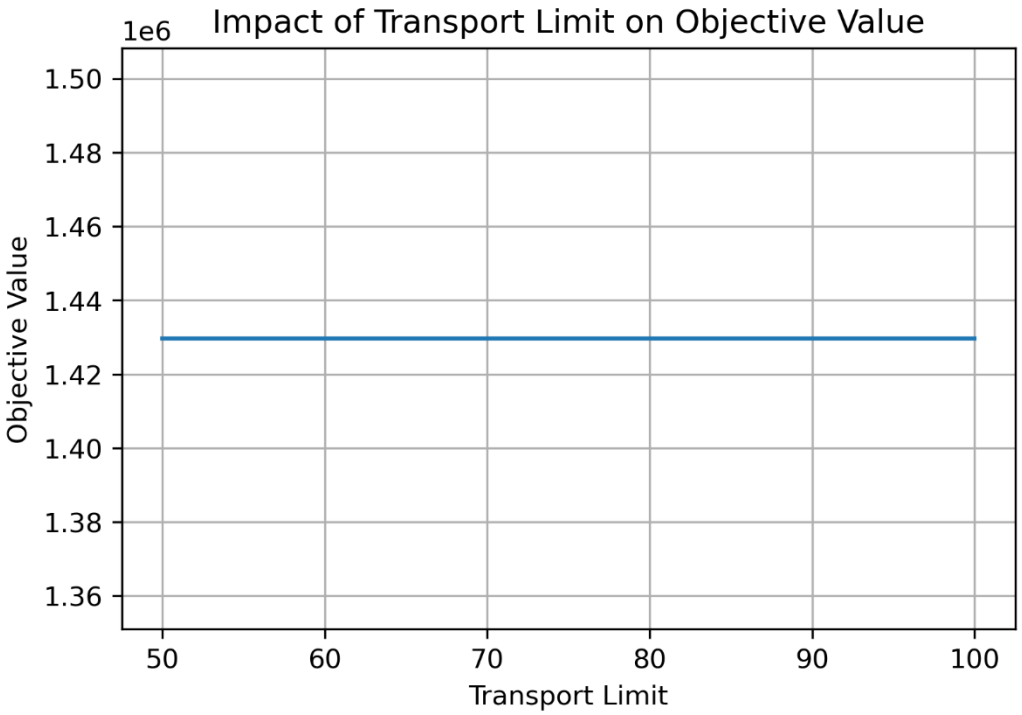
transport_limit بدون حذف قید # تغییر مقدار
for limit in transport_limits_values:
    model.transport_limit.set_value(limit)  # به‌روزرسانی مقدار
    results = solver.solve(model)
    if results.solver.termination_condition == TerminationCondition.optimal:
        objective_values.append(model.Profit.expr())
    else:
        objective_values.append(None)

# حل مدل
    results = solver.solve(model)
    if results.solver.termination_condition == TerminationCondition.optimal:
ذخیره مقدار تابع هدف #
        objective_values.append(model.Profit.expr())
    else:
        objective_values.append(None)  # اگر مدل جواب نداد


x_values=transport_limits_values
y_values=objective_values
if len(x_values) != len(y_values):
تنظیم اندازه کوچکتر #
    min_length = min(len(x_values), len(y_values))
    x_values = x_values[:min_length]
    y_values = y_values[:min_length]

# رسم نمودار
plt.plot(x_values, y_values)
plt.xlabel("Transport Limit")
plt.ylabel("Objective Value")
plt.title("Impact of Transport Limit on Objective Value")
plt.grid(True)
plt.savefig('D:objective_vs_transport_limit.png', dpi=300, bbox_inches='tight')  # تنظیمات ذخیره

plt.show()
```



که طبق چیزی که مشاهده میکنید این مقدار تاثیر بر تابع هدف ندارد چون اصلا مقادیر بهینه ما نزیک 65 نیستند.

قسمت ح)

در این قسمت سعی داریم مقادیر جدول 4 و 5 را با ضرب کردن عددی بین 0.9 تا 1.1 تغییر دهیم

```
model.deal2=Param(initialize=(costContract['2']), mutable=True)
model.transport_change = Param(initialize=1, mutable=True)
# هزینه ها
def cost_rule(model):
    return model.Total_Cost_expr == (
        45 * model.O['1'] + 65 * model.O['2'] + 70 * model.O['3'] +
       model.cost_change* (model.AP['A','0'] + model.AP['B','0']) +
        costBuy['1','A'] * model.AB['A','1'] + costBuy['1','B'] * model.AB['B','1'] +
        costBuy['2','A']* model.AB['A','2'] + costBuy['2','B'] * model.AB['B','2'] +
        model.deal1 * model.C['1'] + model.deal2 * model.C['2'] +
        sum(sum((model.CCS[F, S] ) * TCS[F, S]*model.transport_change
                for S in model.storage_facilities) for F in model.production_facilities) +
        sum(sum((model.CCC[S, C] ) * TCC[S, C]*model.transport_change
                for C in model.cities) for S in model.storage_facilities)
    )
model.Total_Cost_expr = Var(domain=NonNegativeReals, initialize=0)
model.Total_Cost = Constraint(rule=cost_rule)
#-----------------------------------------------------------------------------
```

```
#============================================================
 بخش ح بررسی تغییر #
initial_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
model.transport_change.set_value(model.transport_change.value+0.1 )
solver.solve(model)
with open("D:1.1.txt", "w") as f:
    model.display(ostream=f)
new_basis = {v.name: value(v) for v in model.component_data_objects(Var)}
model.transport_change.set_value(model.transport_change.value -0.2)
solver.solve(model)
with open("D:0.9.txt", "w") as f:
    model.display(ostream=f)
new_basis1 = {v.name: value(v) for v in model.component_data_objects(Var)}
# مقایسه پایه ها
if initial_basis != new_basis:
    print("مجموعه پایه تغییر میکند با 1.1 برابر شدن هزینه ها!")
else:
    print("The basis remains the same.")

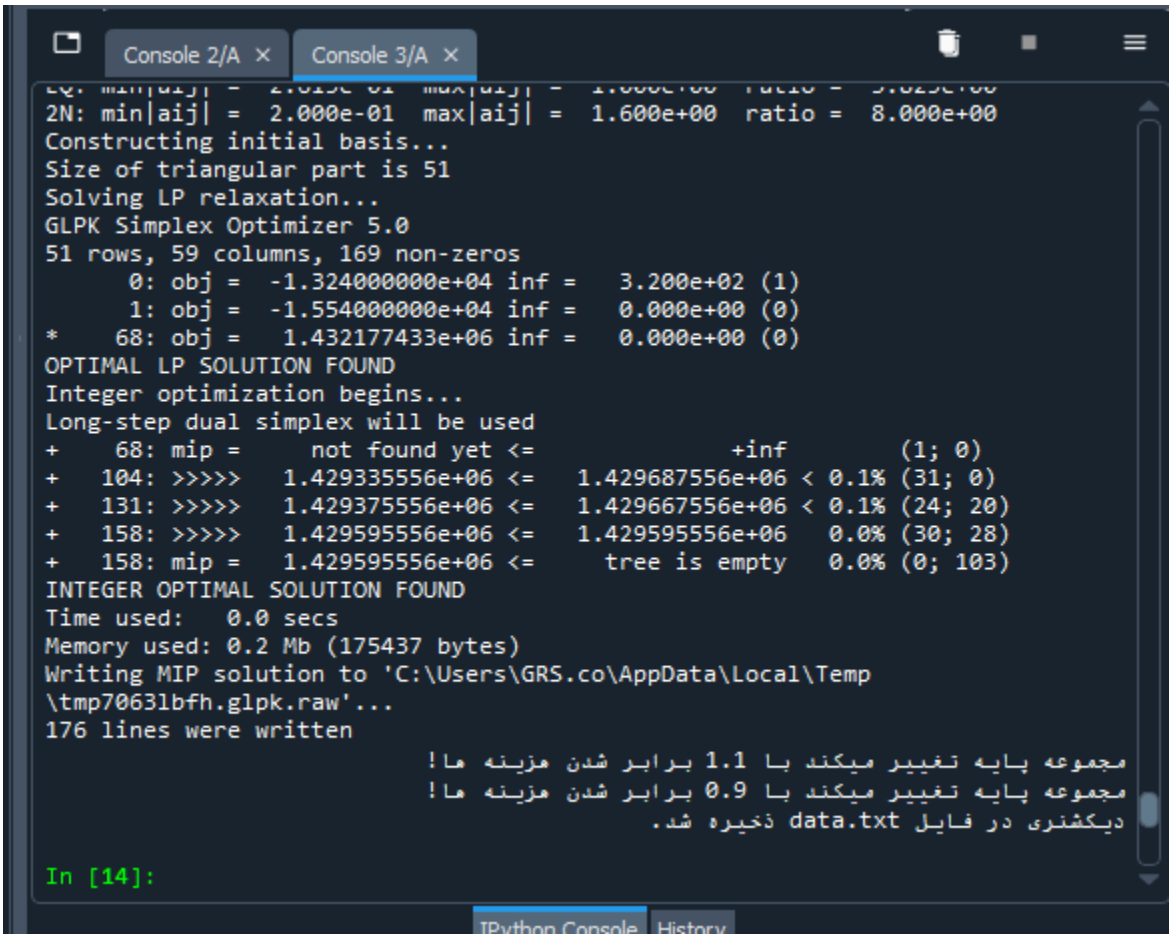
if initial_basis != new_basis1:
    print("مجموعه پایه تغییر میکند با 0.9 برابر شدن هزینه ها!")
else:
    print("The basis remains the same.")

different={j:i-k for i in new_basis.values() for j,k in initial_basis.items()}
with open("D:data.txt", "w", encoding="utf-8") as f:
    for key, value in different.items():
        f.write(f"{key}: {value}\n")

print("دیکشنری در فایل data.txt ذخیره شد.")
#============================================================
```

مقادیر بهینه 0.9 کردن ضرایب در فایل و 1.1 کردن ضرایب در فایل امده اند. همچنین تفاوت مقادیر بهینه در حالت ابتدایی با حالتی که در ان ضرایب 1.1 برابر شده اند نیز در فایل امده است.

در دو حالت مقادیر بهینه عوض میشوند.



برای رسم

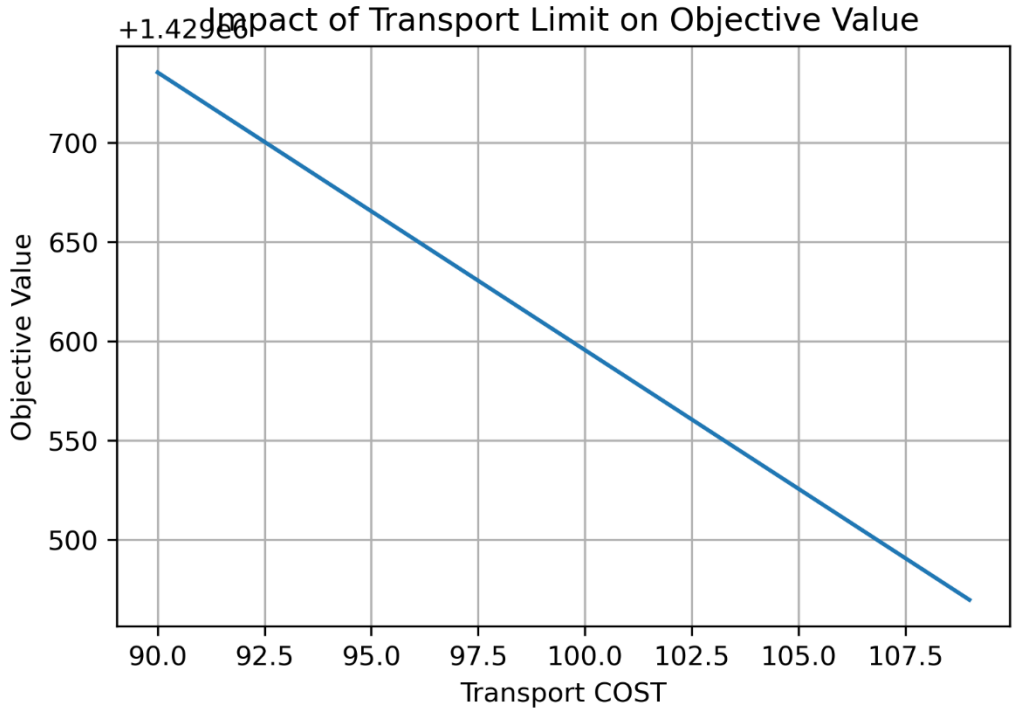
هزینه انتقال از کارخانه صفر به تهران :

با حلقه این تغییرات به دست اورده و در انتها بر روی نمودار نشان میدهیم.

```
#=============================================================================
objective_values=[]
model.transport_change1.set_value(1)

for i in range (90,110):
    model.transport_change2.set_value(i/100)
    results = solver.solve(model)
    objective_values.append(model.Profit.expr())
x_values=transport_limits_values
y_values=objective_values
plt.plot(x_values, y_values)
plt.xlabel("Transport COST")
plt.ylabel("Objective Value")
plt.title("Impact of Transport Limit on Objective Value")
plt.grid(True)
plt.savefig('D:TCS.png', dpi=300, bbox_inches='tight')  # تنظیمات ذخیره

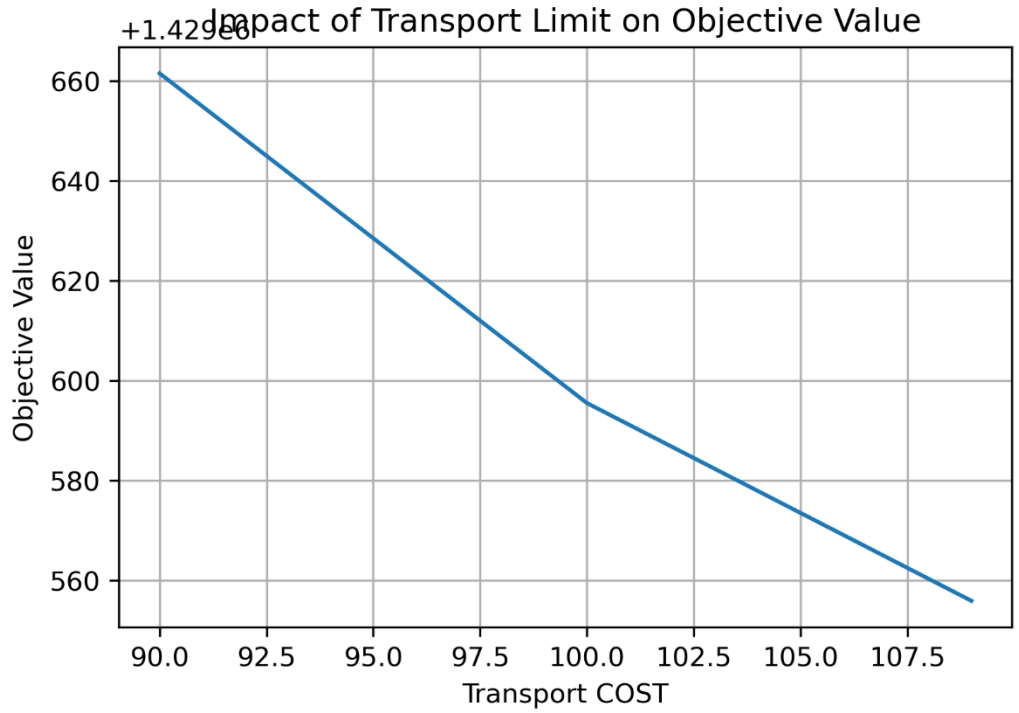
plt.show()
```



هزینه انتقال از تهران به مشهد :

```python
#=============================================================================
import matplotlib.pyplot as plt
transport_limits_values = list(range(90, 110))
model.transport_change.set_value(1)
objective_values=[]
for i in range (90,110):
    model.transport_change1.set_value(i/100)
    results = solver.solve(model)
    objective_values.append(model.Profit.expr())
x_values=transport_limits_values
y_values=objective_values
plt.plot(x_values, y_values)
plt.xlabel("Transport COST")
plt.ylabel("Objective Value")
plt.title("Impact of Transport Limit on Objective Value")
plt.grid(True)
plt.savefig('D:TCC.png', dpi=300, bbox_inches='tight')  # تنظیمات ذخیره

plt.show()
#=============================================================================
```



شیب هزینه انتقال از کارخانه صفر به تهران بیشتر از هزینه انتقال بیشتر از شیب انتقال از انبار تهران به مشهد است پس حساس تر است و نیازمند این است که دقیق تر محاسبه بشود.

قسمت ط)

در این بخش باید شهر ابادان را به کدمان افزوده و بررسی کنیم که چه زمانی با افزایش قیمت فروش، فروش به صرفه و بزرگتر از صفر میشود. این بخش نیز به کمک حلقه انجام میشود.

پیدا کردن کمینه قیمت فروش الیاژ A در ابادان:

```
while True:
# حل مدل
    results = solver.solve(model)
    print(model.AIC['A','T','A'].value+model.AIC['A','E','A'].value,model.ASA.value)
    if model.AIC['A','T','A'].value+model.AIC['A','E','A'].value >0:
        threshold_cost = model.ASA.value
        break


# افزایش cost به مقدار 10 (step واحد)
    model.ASA.set_value(model.ASA.value +10)

# خروجی
print(f"هزینه بحرانی : {threshold_cost}")
D= model.AIC['B','T','A'].value+model.AIC['B','E','A'].value
```

```
0.0 160
0.0 170
0.0 180
0.0 190
0.0 200
0.0 210
0.0 220
0.0 230
0.0 240
0.0 250
0.0 260
0.0 270
0.0 280
0.0 290
0.0 300
0.0 310
0.0 320
0.0 330
0.0 340
0.0 350
0.0 360
0.0 370
100.0 380
هزینه بحرانی : 380

In [16]:
```

پیدا کردن کمینه قیمت فروش الیاژ B در ابادان:

```
#-------------------------------------------------------------------------------
while True:
# حل مدل
    results = solver.solve(model)
    print(model.AIC['B','T','A'].value+model.AIC['B','E','A'].value,model.BSA.value)
    if model.AIC['B','T','A'].value+model.AIC['B','E','A'].value >0:
        threshold_cost = model.BSA.value
        break


# افزایش cost به مقدار 10) step واحد)
    model.BSA.set_value(model.BSA.value +10)

# خروجی
print(f"هزینه بحرانی : {threshold_cost}")
```

```
0.0 330
0.0 340
0.0 350
0.0 360
0.0 370
0.0 380
0.0 390
0.0 400
0.0 410
0.0 420
0.0 430
0.0 440
0.0 450
0.0 460
0.0 470
0.0 480
0.0 490
0.0 500
0.0 510
0.0 520
0.0 530
0.0 540
1306.66666666667 550
                        550 : هزینه بحرانی

In [24]:
```